Daily Progress Report — June 23, 2026

SMA + Stigmergy 3D Drone Swarm Simulation

Summary

Today was the most productive session of the entire project. The algorithm went from a broken state (fitness ~51, wrong coordinates, drones not moving) to a **fully converged result** (fitness **1.780**, final position within 2 metres of target in all three axes). Every major algorithmic bug was identified and fixed.

Session Start State

Item	Status at Start
Both drones arming	❌ COMMAND_DENIED on every attempt
Drone 1 position	❌ Mirroring Drone 0 exactly
Altitude in algorithm	❌ Stuck at boundary (-25.0) every run
Algorithm convergence	❌ Final fitness ~33–51, far from target
Coordinate system	❌ Inconsistent — positive/negative altitude mixed
sma_core.py	❌ Indentation bug, divide-by-zero, wrong contraction

---

Session End State

Item	Status at End
Both drones arming	✅ Both arm reliably every run

| Algorithm convergence | ✅ Final fitness **1.780** |

| Final position | ✅ `[40.44, 41.65, 15.48]` vs target `[40, 40, 15]` |

| Coordinate system | ✅ Fully consistent positive-up altitude |

| sma_core.py | ✅ All 5 bugs fixed |

| settings.py | ✅ Clean, correct bounds and parameters |

| Drone 1 physical flight | ❌ Still mirroring Drone 0 in Gazebo (next session) |

Problems Solved Today

1. COMMAND_DENIED on Arming (Morning)

Symptom: Both drones connected successfully but arming failed on every attempt with `COMMAND_DENIED`.

Root cause: Previous run left PX4 in post-flight state. After landing, PX4 requires a clean restart before accepting new arm commands. Additionally, the `EKF2_MAG_TYPE` heading reference check was blocking arming during fresh starts.

Fix:

- Established mandatory full restart procedure between every run

- Set `param set EKF2_MAG_TYPE 0` in PX4 shell to disable compass heading requirement in simulation

- Added 5-second settle delay after GPS ready before arming attempt

- Added 5-attempt retry loop with 3s gap between attempts

Result: Both drones now arm reliably on first attempt every run.

---

2. Altitude Stuck at Boundary

Symptom: Every `x_best` showed altitude clamped at `-25.0` regardless of target altitude `-15.0`.

Root cause: Coordinate system inconsistency. The algorithm used negative altitude (NED Down convention) but `telemetry.py` was already flipping to positive-up (`-down_m`). So the algorithm was searching in negative space while drones were physically at positive altitude. The boundary `-25.0` was the closest the algorithm could get to target `-15.0` in the wrong coordinate space.

Fix:

- Changed `TARGET` from `[40, 40, -15]` to `[40, 40, 15]` (positive altitude)

- Changed `WORLD_BOUNDS['alt']` from `(-25.0, -5.0)` to `(8.0, 20.0)` (positive range)

- Changed `JAMMED_ZONE` altitude values from negative to positive

- Changed `OBSTACLE_CENTER` altitude from `-12.0` to `12.0`

- Confirmed `setpoint.py` already had the correct outgoing flip (`-target_pos[2]`)

Result: Altitude now explored freely between 8m and 20m, converging to ~15m.

---

3. Five Bugs in sma_core.py

Bug 1 — b decay too aggressive:

```python
# Before (reaches 0 by iteration 100 — kills all contraction range)

b = 1.0 - frac


# After (stays above 0.7 even at iteration 100)

b = 1.0 - frac * 0.3
```

Bug 2 — Indentation error in contraction block:

```python
# Before (comment at wrong indent — Python misparsed the else block)
        else:
    # (c) Contraction behavior
            v_c = ...

# After (correct indentation)
        else:
            # (c) Contraction behavior
            v_c = ...
```

Bug 3 — divide-by-zero in compute_weights:

```python
# Before (caused RuntimeWarning and occasional NaN weights)
log_val = np.log10((f_best - fitness_vals[idx]) / denom + 1)

# After (safe)
ratio   = np.clip((f_best - fitness_vals[idx]) / denom, 0.0, None)
log_val = np.log10(ratio + 1 + 1e-10)
```

Bug 4 — zero step_norm division:

```python
# Before (NaN if step is exactly zero)
step = (step / step_norm) * MAX_STEP

# After (guarded)
if step_norm > 1e-8 and step_norm > MAX_STEP:
    step = (step / step_norm) * MAX_STEP
```

Bug 5 — contraction collapsing toward origin:

```python
# Before (multiplying position by small v_c collapses toward origin)
new_positions[i] = v_c * positions[i]

# After (directed movement toward x_best)
new_positions[i] = positions[i] + v_c * (x_ref - positions[i])
```

---

4. Algorithm Stagnation (Long Flat Sections)

Symptom: Algorithm would find a position then not improve for 20–40 iterations before a random jump rescued it.

Root cause: `Z_RANDOM = 0.03` was too low — only 3% chance of exploration per iteration. With 2 agents effectively behaving as 1 (Drone 1 mirroring), diversity was near zero. Combined with fast `b` decay, the algorithm was in pure contraction mode by iteration 30.

Fix:

- Increased `Z_RANDOM` from `0.03` to `0.15`

- Slowed `b` decay from `1 - frac` to `1 - frac * 0.3`

Result: Algorithm now finds improvements every 5–10 iterations in early phase, converging smoothly.

---

5. QGroundControl Configuration

Symptom: MAVLink Key field in Telemetry settings — user unsure if needed.

Finding: MAVLink 2 signing key is for real-world secure deployments only. Not needed for simulation.

Action: Left empty. Enabled CSV telemetry logging for future paper data collection.

---

6. PX4 Sensor Failure When Launching Gazebo Separately

Symptom: When launching `gz sim` first then PX4 separately, all sensors failed:

```
Preflight Fail: No valid data from Accel 0
Preflight Fail: barometer 0 missing
Preflight Fail: No valid data from Gyro 0
```

Root cause: The Gazebo-PX4 sensor bridge (`gz_bridge`) requires PX4 to launch Gazebo itself to properly initialise sensor topic subscriptions. Launching Gazebo independently breaks the bridge.

Fix: Always launch Drone 0 via `make px4_sitl gz_x500_windy` which starts Gazebo automatically. Drone 1 then joins the already-running Gazebo instance.

---

7. Restart Script Created

To streamline the mandatory between-run restart, created `~/robotics_sim/restart.sh`:

```bash
#!/bin/bash
```

sudo pkill -9 -f px4

sudo pkill -9 -f gz

sudo pkill -9 -f mavsdk

sleep 5

echo "Clean. Ready to launch."

```

---

## Convergence Progress Across All Runs Today

| Run | Final Fitness | Final Position | Key Change Made |
|---|---|---|---|
| Run 1 | 51.479 | [11.97, -1.17, -2.0] | Baseline (broken) |
| Run 2 | 33.770 | [7.98, 36.05, -25.0] | Takeoff altitude fixed to -15m |
| Run 3 | 25.739 | [16.40, 31.01, -25.0] | Staggered arming added |
| Run 4 | 19.999 | [58.39, 38.07, 22.6] | Coordinate system partially fixed |
| **Run 5** | **1.780** | **[40.44, 41.65, 15.48]** | **All fixes applied** ✅ |

---

## Algorithm Performance — Final Run (Run 5)

```
Iteration 1: 56.495 starting at origin

Iteration 8: 27.501 first major jump (-29 fitness in one step)

Iteration 16: 25.573

Iteration 19: 22.222

Iteration 23: 11.450 entered target region

Iteration 25:   5.177 within 5m of target

Iteration 29:   3.897

Iteration 36:   3.259

Iteration 37:   2.955

Iteration 38:   2.614

Iteration 39:   2.263

Iteration 42:   1.823

Iteration 43:   1.780 ← converged (held for iterations 43–100)

```

**Convergence achieved at iteration 43 out of 100.**

**Error at convergence: North +0.44m, East +1.65m, Alt +0.48m**

---

## Remaining Issue for Next Session

### Drone 1 Not Moving Physically in Gazebo

**Symptom:** Both drone objects arm, take off, and receive setpoints — but only Drone 0 physically moves in Gazebo. Drone 1 hovers at takeoff position for all 100 iterations.

**Evidence:**

```
Drone 0 raw position: N=0.074 E=0.028 Alt=15.004

Drone 1 raw position: N=0.075 E=0.026 Alt=15.004 ← identical to Drone 0
```

**Suspected cause:** MAVSDK telemetry subscription for Drone 1 is reading from the same PX4 MAVLink stream as Drone 0. The setpoints sent to Drone 1 may also be going to Drone 0 silently.

**Next session plan:**

1. Read `telemetry.py` and `main.py` in full

2. Verify drone object independence — check if MAVSDK `System()` instances are truly separate
```

3. Verify setpoints are being sent to correct drone instance

4. If telemetry is confirmed cross-reading, implement per-drone position verification before algorithm starts

---

Environment Reference

Component	Version / Detail
OS	WSL2 Ubuntu 22.04 on Windows 11
PX4	v1.14 SITL
Gazebo	Harmonic 8.x
Python	3.14
MAVSDK	2.x
World	windy.sdf
Drone model	x500
Drone 0 port	14540
Drone 1 port	14541
Drone 0 spawn	`[0, 0, 0.1]`
Drone 1 spawn	`[10, 0, 0.1]`

Files Modified Today

File	Changes
`config/settings.py`	Fixed coordinate system, tightened bounds, increased Z_RANDOM
`algorithm/sma_core.py`	Fixed 5 bugs (b decay, indentation, divide-by-zero, step guard, contraction)

| `drone_control/connection.py` | Added GPS timeout, arming retry loop, settle delay, offboard retry |

| `main.py` | Changed parallel arming to staggered sequential, added position debug print |

| `~/robotics_sim/restart.sh` | New file — clean restart script |

Report generated: June 23, 2026

Next session: Fix Drone 1 independent flight in Gazebo